CP312 ASSIGNMENT 5

I certify that this assignment is entirely my own work and that all external sources, if used, have been properly acknowledged.

1.

To find the shortest cycle containing vertex v in an undirected graph G=(V,E) we use a modified Breadth First Search. BFS is optimal here because it explores edges in increasing order of distance from the source.

↳ Initalize: Start a BFS from vertex V. Maintain a dist[] array where dist[v] =0 and all others are ∞, and a parent[] array to track the BFS tree

↳ BFS Traversal: For each vertex U dequeued, iterate through its neighbours W

↳ If W is unvisited: Set dist[w] = dist[v] +1, Parent[w] = U, and enqueue W.

↳ If W is already visited and W=V and U ≠ Parent[v] (where v has no Parent): a cycle is found. The length is dist[v] +1

↳ Special case: if we encounter a visited vertex w that is already in the queue (and parent[v] ≠w), we have found a cycle. If v is part of the path to both U and W, the cycle length is dist[v] +dist[w] +1

↳ Termination: The first cycle found that includes v is guaranteed to be the shortest due to the layer by layer nature of BFS.

Pseudocode:

```
Shortest(ycle(G,V):
    Queue Q
    dist[V] = {infinity}, Parent[V] = {null}
    dist[v] = 0
    Q.enqueue(v)

    min_cycle = infinity

    While Q is not empty:
        U = Q.dequeue()
        for each neighbor W of U:
            if dist[w] == infinity:
                dist[w] = dist[u] +1
            Parent[w] = U
                Q.enqueue(w)
            else if W==V and U != Parent[w]:
                return dist[u] + 1   //shortest cycle found
    return "No cycle exists"
```

Complexity Analysis:

Time: Each vertex is enqueued once and each edge is processed at most twice (once from each endpoint). This results in a total time complexity of $O(V+E)$

Space: $O(V)$ to store the dist, Parent and Queue

2.

In a tree with V vertices and |E| = |V| - [cites start] 1, removing a single vertex v breaks the tree into a smaller number of connected components equal to the degree of vertex v

↳ removing a leaf (degree 1) leaves component 1.

↳ removing a internal node with k neighbors leaves k components.

↳ To maximize the number of components, we must find the vertex with the maximum degree.

Pseudocode:

MaxComponents(Tree T):

Input: Adjacency list adj[] for the tree

Output: Maximum number of components after removing one vertex

```
max_count = 0
for i from 0 to V-1:
    current_degree = length(adj[i])
    if current_degree > max_count:
        max_count = current_degree
return max_count
```

Example walkthrough:

- Vertex 2 is connected to vertices 1 and 3
- Vertex 1 is connected to 0 and 2
- Vertex 3 is connected to 2 and 4
- removing vertex 1, 2, or 3 results in 2 components (max for the specific path graph)

Time complexity:

Time: Algorithm iterates through adj list once. Sum of all degrees in any graph is 2|E|. Since |E| = |V| - |cites start| 1 in a tree, complexity is O(V)

Space: O(1) additional space (excluding the input tree storage